Relatório Técnico

Disciplina: DIM0501 - Boas Práticas de Programação
Tema: Aplicação dos conceitos de boas práticas em cenário simulado
Semestre: 2026.1

1. Identificação do Grupo

- **Integrante 1:** Gabriel Eugênio Vitalino da Silva
- **Integrante 2:** Helder Pereira Sá Paixão
- **Integrante 3:** José Francisco de Souza Fonseca
- **Integrante 4:** Matheus Henrique Ferreira da SIlva
- **Linguagem utilizada:** Python
- **Link do repositório Git:** https://github.com/josefrancisco341/safe-news-check

2. Descrição do Sistema

O sistema, batizado de **Safe News Check**, é uma aplicação de console em Python para o **Monitoramento de Fake News e Qualidade da Informação**.

- **O que o sistema faz:** cadastrar textos de notícias, classificá-los automaticamente como confiável, duvidosa ou falsa com base em critérios textuais (ausência de fonte, uso de linguagem sensacionalista como !!! e URGENTE e tamanho do texto), além de listar todas as notícias cadastradas.
- **Objetivo do código original:** oferecer um protótipo simples de triagem de notícias, simulando a etapa inicial de uma ferramenta de checagem de fatos em que um analista insere manualmente o conteúdo a ser avaliado.
- **Contexto de utilização:** pode ser empregado em ambientes educacionais (aulas de combate à desinformação), em redações jornalísticas como apoio à curadoria, ou como base para sistemas maiores de verificação automatizada de conteúdo.

3. Problemas Identificados

A versão original (arquivo único Sistema.py), apesar de funcional, apresenta diversos problemas de qualidade que dificultam a manutenção e a evolução do código.

3.1 Problemas de Legibilidade

- Nomes de variáveis e funções extremamente curtas e pouco descritivas (f, func2, d, a, b, t, c).
- Uso de chaves "mágicas" em dicionários ("t", "c") que não comunicam o que armazenam.
- Comparação com None feita com == em vez do idiomático is None.
- Mensagens ao usuário sem padronização e sem acentuação ("erro", "errado").

Trecho do código original:

```python
def f(a, b=None):
    # essa função adiciona uma coisa
    if a != "":
        d = {}
        d["t"] = a
        if b == None:
            d["c"] = "duvidosa"
        else:
            d["c"] = b
        data.append(d)
    else:
        print("erro")
```

Sem ler todo o programa, é impossível saber o que f, a, b, d["t"] ou d["c"] representam.

3.2 Problemas de Organização

- Todo o sistema é concentrado em **um único arquivo** (Sistema.py), misturando modelo de dados, regra de negócio (classificação), persistência em memória e interface com o usuário.
- A função f acumula múltiplas responsabilidades: válida entrada, monta o registro, define classificação padrão e persiste.
- A função menu() é chamada diretamente no topo do script, sem guarda `if __name__ == "__main_ _":`, impedindo o reaproveitamento do módulo em testes ou em outros programas.
- Uso de for in range(0, len(data)) em vez de iteração direta sobre a coleção, denotando estilo procedural fora dos idiomas da linguagem.

3.3 Código Duplicado ou Design Ruim

- Existem **dois caminhos paralelos** para adicionar uma notícia (add manual e add_auto), e ambos terminam chamando f com lógicas levemente diferentes - a classificação padrão "duvidosa" aparece tanto em f quanto implicitamente em add manual.
- A função analisar retorna strings literais ("confiável", "duvidosa", "falsa") repetidas em vários pontos do código, sem uma única fonte da verdade.
- A função f recebe um parâmetro opcional b que é tratado como classificação, mas o nome não indica isso - design confuso que mistura "adicionar com classificação manual" e "adicionar com classificação automática" na mesma assinatura.
- A variável global da data é manipulada diretamente por várias funções, sem qualquer encapsulamento.

3.4 Falta de Validação

- A única validação de entrada é if a != "", que aceita strings compostas apenas por espaços em branco ("") como notícias válidas.
- Não há tratamento de exceções para entradas inesperadas.
- O fluxo de erro simplesmente imprime "erro" e segue adiante, sem informar ao chamador que a operação falhou (a função não retorna nada que permita modificar o comportamento).

- A classificação manual aceita qualquer string fornecida pelo usuário, mesmo que não seja confiável, duvidosa ou falsa.

3.5 Problemas de Documentação

- Comentários redundantes e inúteis:
  - `# sistema de noticias no topo do arquivo;`
  - `# função que faz tudo (não descreve nada);`
  - `# essa função adiciona uma coisa;`
  - `# lista tudo;`
  - `# analisa o texto;`
  - `# comentários desnecessários abaixo, # inicia o programa, # chama o menu.`
- Ausência de docstrings nas funções.
- Ausência de um README explicando o propósito do sistema, suas regras e como executá-lo.
- Critérios de classificação não documentados em lugar nenhum - só podem ser inferidos lendo o corpo de analisar.

4. Estratégias de Solução

4.1 Refatoração

- **Renomeações para nomes descritivos:**
  - f → adicionar notícia
  - func2 → listar notícias
  - analisar → classificar notícia
  - data → noticias
  - Chaves de dicionário "t" / "c" → "texto" / "classificação"
  - Variáveis locais a, b, t, c, d, txt, score → texto, classificação, texto tratado, pontos.
- **Quebra de funções por responsabilidade única:** a antiga f foi decomposta em validar texto (responsável apenas por validar) e adicionar_noticia (responsável apenas por persistir).
- **Eliminação de caminhos duplicados:** as funções add_manual e add_auto foram unificadas em um único fluxo opcao_adicionar, que sempre classifica automaticamente - alinhado ao propósito do sistema (checagem automatizada).
- **Substituição do encadeamento if/elif** do menu pelo construtor match/case do Python 3.10+, mais legível.
- **Guard clause `if __name__ == "__main__":`** adicionada para permitir que o módulo seja importado sem efeitos colaterais.

4.2 Modularização

O arquivo único foi separado em quatro módulos com responsabilidades distintas, seguindo a separação **modelo / serviço / interface / ponto de entrada** sugerida nas diretrizes:

Módulo	**Responsabilidade**
dados.py	Modelo de dados - armazena a lista de notícias (simula o banco de dados).
validador.py	Regra de negócio - classifica uma notícia confiável/duvidosa/falsa.
gerenciador.py	Serviço - valida texto, adiciona e lista notícias.
main.py	Interface - menu interativo e ponto de entrada do programa.

A dependência flui em sentido único (main → gerenciador → dados e main → validador), evitando ciclos.

4.3 Programação Defensiva

- **Validação reforçada de entrada:** a função validar texto rejeita strings vazias **e** strings compostas apenas por espaços em branco (texto.strip() == ""), e devolve o texto já normalizado (sem espaços nas pontas).
- **Tratamento de erro com exceção:** em vez de imprimir "erro" silenciosamente, validar texto levanta ValueError, e adicionar notícia captura essa exceção em um bloco try/except, exibindo uma mensagem clara ao usuário e retornando False - permitindo que o chamador saiba que a operação não foi concluída.
- **Normalização defensiva** no classificador: o texto é convertido para maiúsculas (.upper()) antes das comparações, evitando que variações de caixa (urgente, Urgente, URGENTE) escapem da análise.

- **Listagem segura:** listar notícias trata explicitamente o caso da lista vazia e informa "Nenhuma notícia cadastrada." em vez de simplesmente não imprimir nada.
- **Menu robusto:** o caso default (case _) instrui o usuário sobre as opções válidas, em vez do genérico "errado".

4.4 Documentação

- **Remoção de todos os comentários inúteis** (# sistema de notícias, # função que faz tudo, # lista tudo, etc.).
- **Adição de docstrings** nas funções cuja regra de negócio não é trivial, especialmente classificar notícia, que passa a documentar de forma explícita os critérios de pontuação e os possíveis retornos.
- **Comentários úteis** preservados ou adicionados apenas onde explicam o porquê (ex.: # Normaliza o texto para comparação na linha do .upper()).
- **Criação de um arquivo readme.md** contendo: descrição do projeto, funcionalidades, tabela com regras de classificação, tabela com a classificação final, pré-requisitos e instruções de execução em Linux/macOS, Windows CMD e Windows PowerShell.

5. Exemplos de Melhoria (Antes vs Depois)

Exemplo 1 - Função de inserção de notícia

Antes:

```python
data = []

def f(a, b=None):
    # essa função adiciona uma coisa
    if a != "":
        d = {}
        d["t"] = a
        if b == None:
            d["c"] = "duvidosa"
        else:
            d["c"] = b
        data.append(d)
    else:
        print("erro")
```

Depois:

```python
# dados.py
noticias = []
# gerenciador.py

from dados import noticias

def validar_texto(texto):
    #Levanta ValueError se o texto for vazio ou só espaços.
    if not texto or texto.strip() == "":
        raise ValueError("Texto da notícia não pode ser vazio.")
    return texto.strip()

def adicionar_noticia(texto, classificacao):
    #Adiciona notícia. Retorna True se o texto for válido ou False se for vazio.
    try:
        texto_valido = validar_texto(texto)
        noticias.append({"texto": texto_valido, "classificacao": classificacao})
        return True
    except ValueError as e:
        print(f"Erro: {e}")
        return False
```

Explicação: A função f tinha responsabilidades misturadas (validar entrada, criar registro, definir classificação padrão e persistir) e seus parâmetros (a, b) e chaves ("t", "c") não comunicavam nada. Na versão refatorada:

1. **Responsabilidade única**: validar texto válido; adicionar noticia persiste.
2. **Nomes descritivos** em parâmetros e chaves (texto, classificação).
3. **Validação mais robusta**: rejeita também strings de só espaços via strip().
4. **Tratamento de erro idiomático**: levanta ValueError em vez de imprimir "erro", e o chamador recebe True/False para decidir o próximo passo.
5. **Separação de módulos**: o modelo de dados (notícias) vive em dados.py, fora da camada de serviço.

Exemplo 2 - Menu interativo e fluxos de adição

Antes:

```python
def add_manual():
    t = input("Digite o texto: ")
    c = input("Digite classificacao: ")

    if c == "":
        f(t)
    else:
        f(t, c)

def add_auto():
    t = input("Digite o texto: ")
    c = analisar(t)
    f(t, c)


def menu():
    while True:
        print("1 - adicionar manual")
        print("2 - adicionar automatico")
        print("3 - listar")
        print("4 - sair")

        op = input("opcao: ")

        if op == "1":
            add_manual()
        elif op == "2":
            add_auto()
        elif op == "3":
            func2()
        elif op == "4":
            break
        else:
            print("errado")

# comentarios desnecessarios abaixo
# inicia o programa
# chama o menu
menu()
```

Depois:

```python
# main.py

from validador import classificar_noticia
from gerenciador import adicionar_noticia, listar_noticias

def opcao_adicionar():
    #Lê o texto, classifica automaticamente e adiciona.
    texto = input("\nDigite o texto da notícia: ")
    classificacao = classificar_noticia(texto)
    if adicionar_noticia(texto, classificacao):
        print(f"Notícia adicionada! Classificação: {classificacao}")

def opcao_listar():
    listar_noticias()

def menu():
    while True:
        print("\n===== SAFE NEWS =====")
        print("1 - Verificar confiabilidadade de notícia")
        print("2 - Listar notícias")
        print("3 - Sair")
        opcao = input("Digite a opção desejada: ")

        match opcao:
            case "1":
                opcao_adicionar()
            case "2":
                opcao_listar()
            case "3":
                print("Saindo...")
                break
            case _:
                print("Opção inválida. Digite 1, 2 ou 3.")

if __name__ == "__main__":
    menu()
```

Explicação:

1. **Eliminação de código duplicado**: os dois fluxos paralelos (`add_manual` e `add_auto`) foram unificados em `opcao_adicionar`. A classificação manual era um vetor de inconsistência (o usuário podia digitar qualquer string como classificação) e contradiz o propósito do sistema - checagem automática de notícias.

2. **Uso de match/case** (Python 3.10+) em vez de if/elif, deixando o despacho de opções mais expressivo.
3. **Mensagens de menu mais informativas e padronizadas**, com acentuação correta e título do programa.
4. **Caso padrão útil**: em vez de "errado", o usuário recebe instrução do que pode digitar.
5. **Guarda if __name__ == "__main__":** evita que o menu seja executado por acidente em um import, permitindo testes e reuso.
6. **Imports explícitos**: o main.py declara claramente de quais módulos depende, tornando o grafo de dependências evidente.

6. Organização Final do Código

Estrutura final do projeto:

```
safe-news-check/
├── dados.py          # modelo
├── validador.py      # serviço (regra de negócio: classificação)
├── gerenciador.py    # serviço (validação e persistência em memória)
├── main.py           # interface (menu) e ponto de entrada
└── readme.md         # documentação do projeto
```

Papel de cada parte:

- **dados.py (modelo):** mantém a estrutura de dados que representa o "banco" em memória - uma lista de notícias. Único ponto que conhece o formato físico dos registros.
- **validador.py (serviço - regra de negócio):** contém classificar notícia, função pura que recebe um texto e devolve sua classificação. Não depende de I/O nem de estado global, podendo ser testada isoladamente.
- **gerenciador.py (serviço - operações sobre o modelo):** orquestra a validação do texto, a inserção em dados,notícias e a listagem. É a camada que conecta o domínio (validador) ao modelo (dados).
- **main.py (interface):** apresenta o menu, captura entradas do usuário e delega às funções de serviço. Não conhece detalhes do modelo nem da regra de classificação.

- **readme.md (documentação):** descreve o propósito, as regras de classificação, os pré-requisitos e como executar o projeto em diferentes sistemas operacionais.

A dependência segue o sentido **interface** → **serviço** → **modelo**, sem ciclos.

7. Conclusão

O que o grupo aprendeu com o trabalho: A atividade evidenciou na prática que um código pode "funcionar" e ainda assim ser ruim. Pequenas decisões como nomear corretamente, separar responsabilidades, validar com strip() em vez de comparar com string vazia, escolher entre lançar exceção ou retornar False, têm efeito multiplicador sobre legibilidade e manutenção. Aprendemos que modularizar
Aprendemos também que modularizar não é apenas dividir em arquivos: é estabelecer um sentido claro de dependências entre camadas (modelo, serviço, interface), e isso só fica visível depois que tentamos importar um módulo sem trazer todo o resto junto.

Maiores dificuldades: A principal foi resistir à tentação de reescrever do zero, restrição explícita das diretrizes. O código original tinha problemas em todas as dimensões avaliadas, e, em vários momentos, foi mais fácil "começar de novo" do que renomear, mover e dividir incrementalmente. Outra dificuldade foi decidir o grão certo de modularização: separar validador.py de gerenciador.py foi uma escolha de design, poderiam ter sido um único módulo de serviços, mas a função de classificação é pura e independente, e isolá-la abre caminho para testes unitários e para trocar a regra de classificação no futuro sem mexer no resto.

Como as boas práticas impactaram o código: O Safe News Check passou de um único arquivo de 87 linhas com nomes obscuros, lógica duplicada, validação frágil e comentários inúteis para quatro módulos coesos, com nomes que falam por si, validações que rejeitam entradas inválidas em vez de aceitá-las silenciosamente, e um README que permite a qualquer pessoa executar o projeto sem precisar perguntar. O comportamento externo foi preservado (e até melhorado, no caso de strings só com espaços), mas o custo de evoluir o sistema caiu drasticamente: incluir um novo critério de classificação hoje exige alterar apenas validador.py; trocar a persistência por um banco de dados real exige alterar apenas dados.py e gerenciador.py. É a evidência mais concreta de que boas práticas pagam dívidas técnicas.